

MoMo SMS Analytics

Database Design Document — Week 2

Project: Enterprise MoMo SMS data processing system

Author: Endahiro (solo contributor — extension granted)

Target DBMS: MySQL 8.0+ (verified on MariaDB 10.11)

Deliverables in this document: ERD, design rationale, data dictionary, sample queries with screenshots, security & accuracy rules with proof.

This project is being completed individually due to a registration timing issue. All "team" requirements are fulfilled by a single contributor with the instructor's knowledge.

1. Introduction

The MoMo SMS data processing system ingests Mobile Money confirmation SMS messages exported as XML, cleans and categorises them, and stores structured records in a relational database. That database is the foundation for the analytics dashboard and REST API delivered in later assignments. This document describes the schema, explains the design choices, and shows the database in action against the provided dataset of 1,691 SMS records.

1.1 Dataset shape

Parsing the provided modified_sms_v2.xml file identified eight distinct transaction categories. The distribution of the 1,691 records is: Payment to Code Holder (700), Transfer to Mobile (585), Bank Deposit (248), Incoming Money (63), Direct Payment (36), Airtime Purchase (15), Withdrawal (3), Internet Bundle (1), plus 40 non-transaction messages (OTP notifications) which are excluded from the fact table and logged instead.

2. Entity Relationship Diagram

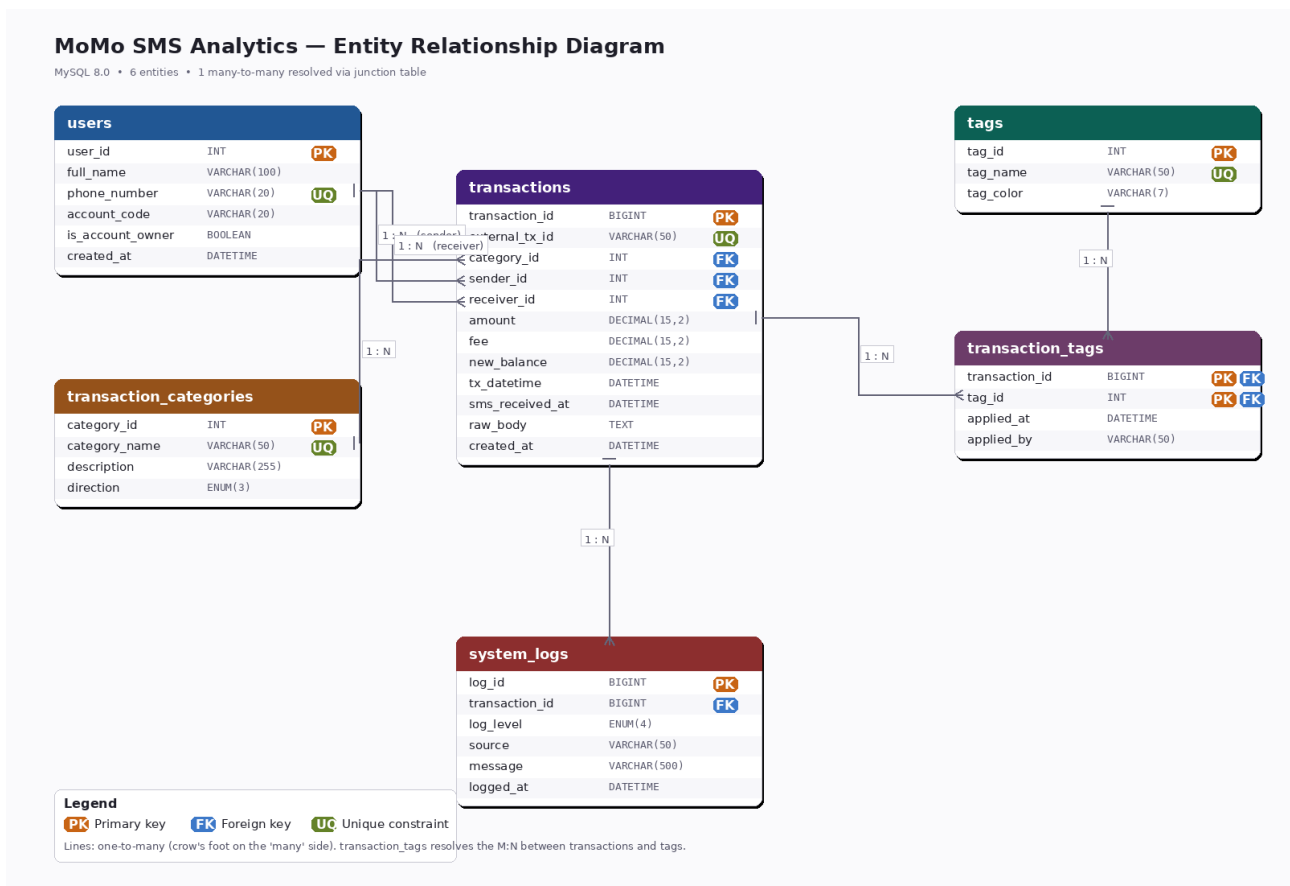


Figure 1 — Full ERD showing all six entities, primary and foreign keys, unique constraints, and cardinalities.

3. Design Rationale

The schema centres on a single fact table, **transactions**, surrounded by lookup and dimension tables. This is a classic star-adjacent design chosen because the dashboard's queries all pivot on the transaction record — grouping by category, summing amounts, filtering by date. Placing categories, users, and tags in separate tables normalises repeated values and keeps the fact table narrow and fast to scan.

Users are normalised. The same customer names recur across dozens or hundreds of SMS. Storing a name once in the **users** table and referencing it by `user_id` from both the sender and receiver columns saves storage and, more importantly, prevents silent drift (e.g. *Jane Smith* vs *Jane Smith*). It also makes it easy to add richer person data later — verification status, customer segment, KYC flags — without touching the transaction table.

Categories are a fixed lookup with a direction flag. Every category is labelled CREDIT, DEBIT, or NEUTRAL. This is more useful than storing direction on each transaction because the direction is an inherent property of the category — an Airtime Purchase is always a DEBIT — and enforcing it centrally prevents inconsistent data.

The M:N requirement is met by the tags/transaction_tags pair. A transaction can be flagged with multiple tags (a merchant payment over 10,000 RWF is both *high_value* and *merchant*), and any tag applies to many transactions. The junction table uses a composite primary key (`transaction_id`, `tag_id`) plus `applied_at` and `applied_by` for audit — so we know when and by whom (or by which automated rule) each tag was attached.

System logs link back to transactions optionally. During ETL, many log entries are transaction-specific ('inserted', 'normalised', 'trailing whitespace'). Others are global ('run started', 'skipped 40 OTP messages'). A nullable FK captures both cases in one table.

Referential integrity is layered. Foreign keys prevent orphan transactions. CHECK constraints catch impossible amounts and malformed data before it lands. Triggers enforce the sender-must-differ-from-receiver rule portably across MySQL and MariaDB. Indexes on `tx_datetime`, `category_id`, `sender_id`, `receiver_id`, and `amount` cover the queries the dashboard will actually run.

4. Data Dictionary

4.1 users

Column	Type	Key	Description
user_id	INT AUTO_INCR	PK	Surrogate key
full_name	VARCHAR(100)		Person name as parsed from SMS
phone_number	VARCHAR(20)	UQ	MSISDN; nullable — some SMS mask it
account_code	VARCHAR(20)		Optional agent/short code
is_account_owner	BOOLEAN		TRUE for the phone owner whose SMS were exported
created_at	DATETIME		Row insertion timestamp

4.2 transaction_categories

Column	Type	Key	Description
category_id	INT AUTO_INCR	PK	Surrogate key
category_name	VARCHAR(50)	UQ	e.g. 'Bank Deposit', 'Airtime Purchase'
description	VARCHAR(255)		What SMS map to this category
direction	ENUM(CREDIT/DEBIT/NEUTRAL)		Money in, money out, or system

4.3 transactions

Column	Type	Key	Description
transaction_id	BIGINT AUTO_INCR	PK	Surrogate key
external_tx_id	VARCHAR(50)	UQ	Financial Transaction Id from SMS body
category_id	INT	FK	→ transaction_categories
sender_id	INT	FK	→ users (nullable: bank deposits have no sender)
receiver_id	INT	FK	→ users (nullable: withdrawals have no receiver)
amount	DECIMAL(15,2)		Amount in RWF; CHECK > 0
fee	DECIMAL(15,2)		MoMo fee in RWF; CHECK ≥ 0
new_balance	DECIMAL(15,2)		Balance after transaction; CHECK ≥ 0
tx_datetime	DATETIME		When the transaction occurred
sms_received_at	DATETIME		When the SMS was received
raw_body	TEXT		Original SMS body kept for audit
created_at	DATETIME		Row insertion timestamp

4.4 tags

Column	Type	Key	Description
tag_id	INT AUTO_INCR	PK	Surrogate key
tag_name	VARCHAR(50)	UQ	e.g. 'high_value', 'merchant'
tag_color	VARCHAR(7)		Hex color for dashboard UI; CHECK regex

4.5 transaction_tags (M:N junction)

Column	Type	Key	Description
transaction_id	BIGINT	PK, FK	→ transactions; part of composite PK
tag_id	INT	PK, FK	→ tags; part of composite PK
applied_at	DATETIME		When the tag was attached
applied_by	VARCHAR(50)		'system', a rule id, or analyst name

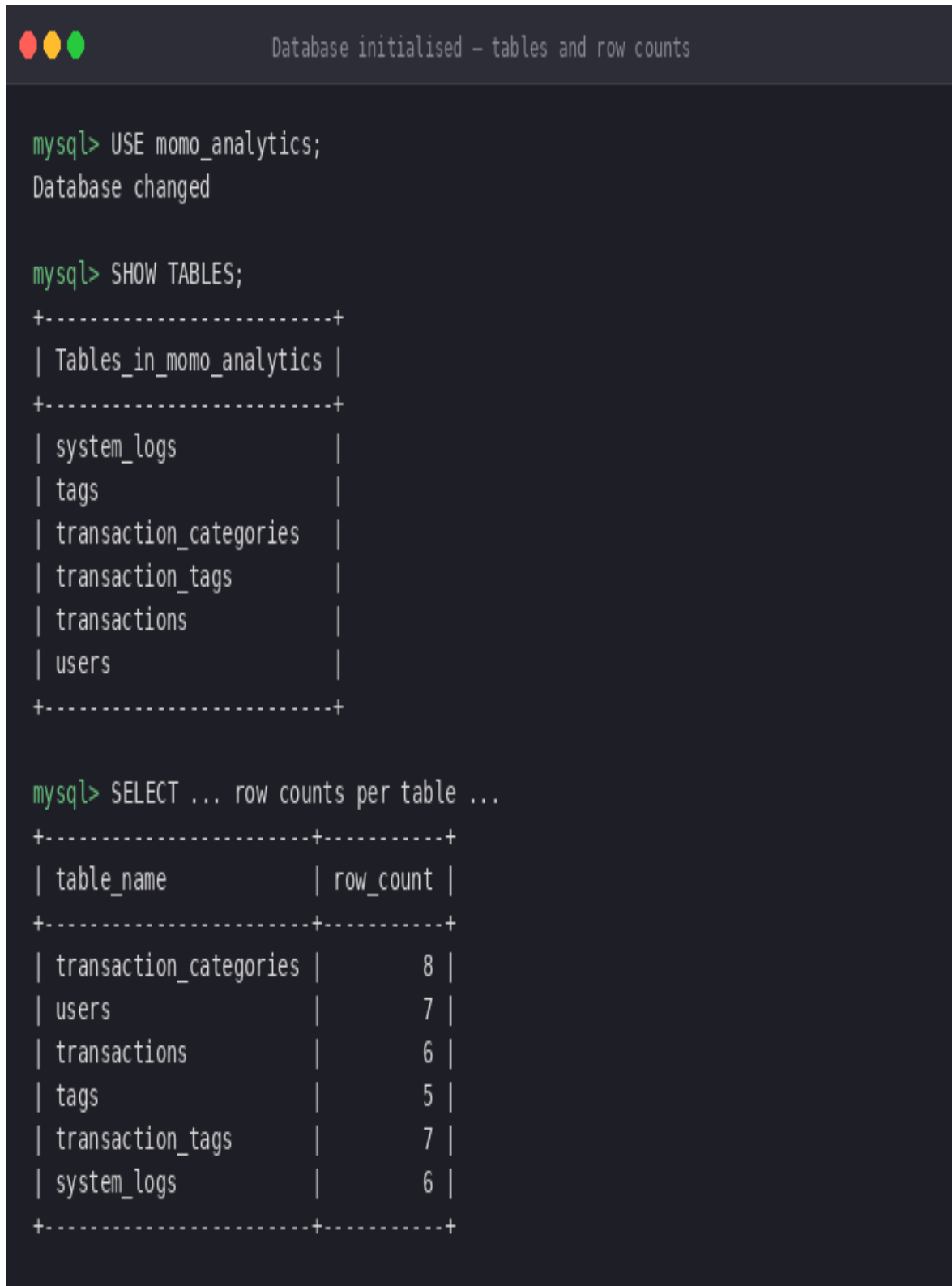
4.6 system_logs

Column	Type	Key	Description
log_id	BIGINT AUTO_INCR	PK	Surrogate key
transaction_id	BIGINT	FK	→ transactions (nullable: global events)
log_level	ENUM(INFO/WARNING/ERROR/DEBUG)		Severity
source	VARCHAR(50)		ETL component (parser, cleaner, loader)
message	VARCHAR(500)		Human-readable message
logged_at	DATETIME		Log timestamp

5. Sample Queries — Executed Against the Live Database

The setup script ends with five queries used for CRUD testing and dashboard prototyping. Each was executed against the live database populated by the sample INSERTs. Screenshots below are captured directly from the MariaDB CLI running the script.

5.0 Database initialised



```
mysql> USE momo_analytics;
Database changed

mysql> SHOW TABLES;
+-----+
| Tables_in_momo_analytics |
+-----+
| system_logs              |
| tags                     |
| transaction_categories   |
| transaction_tags         |
| transactions             |
| users                    |
+-----+

mysql> SELECT ... row counts per table ...
+-----+-----+
| table_name          | row_count |
+-----+-----+
| transaction_categories |          8 |
| users                |          7 |
| transactions         |          6 |
| tags                 |          5 |
| transaction_tags     |          7 |
| system_logs          |          6 |
+-----+-----+
```

Figure 2 — All six tables created and populated.

5.1 All transactions with names (multi-table JOIN)

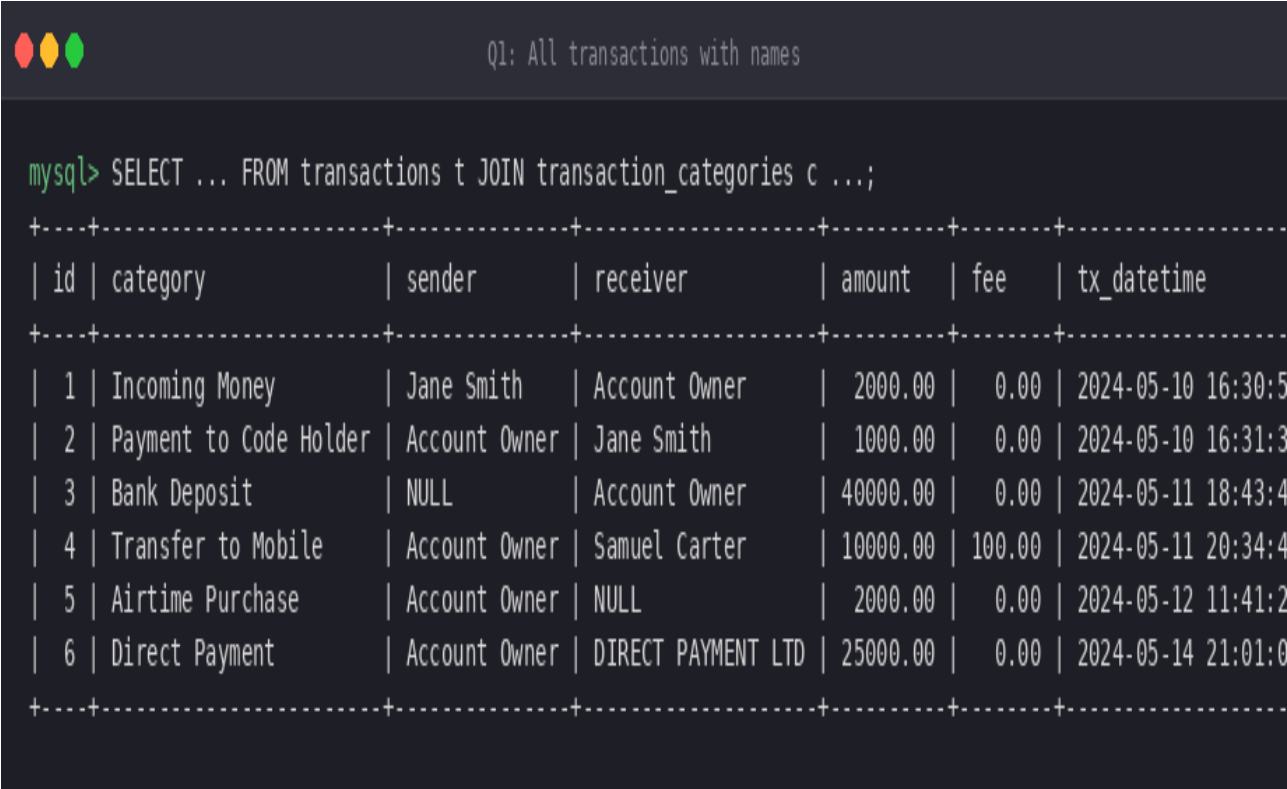


Figure 3 — Every transaction with its category and party names resolved via LEFT JOINS to users (senders and receivers are nullable).

5.2 Volume per category (dashboard aggregate)

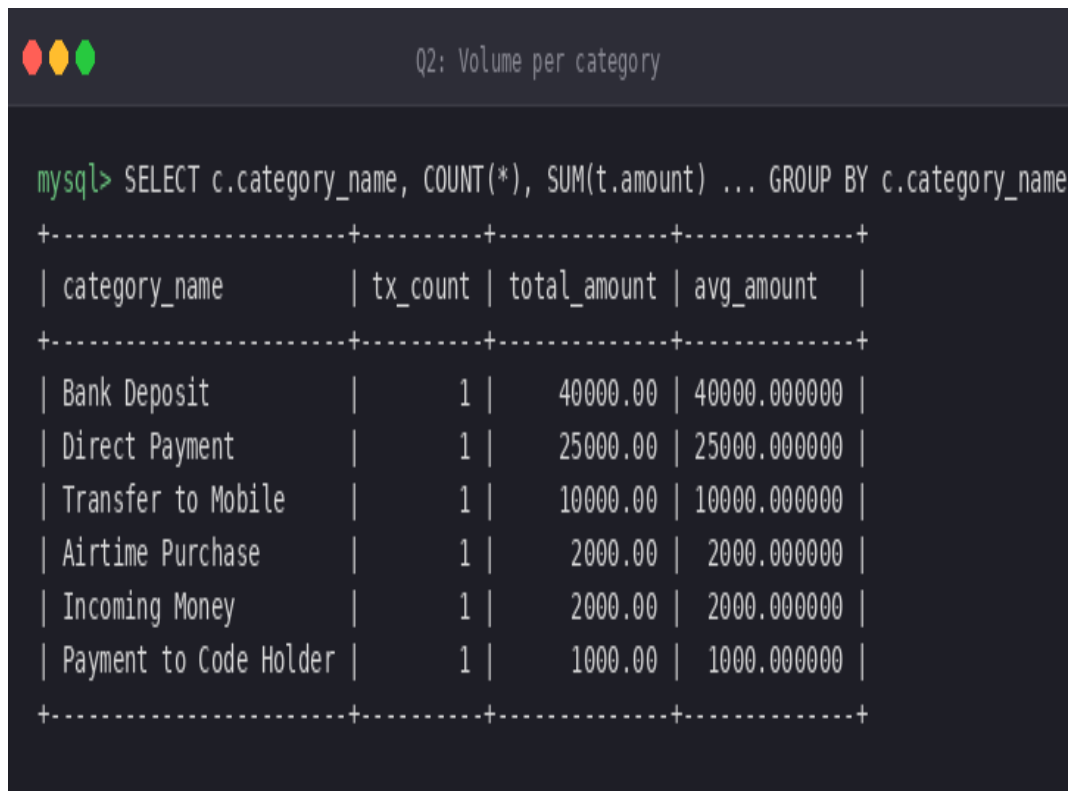


Figure 4 — GROUP BY producing the numbers the dashboard's pie chart will consume.

5.3 High-value transactions via the M:N tag join

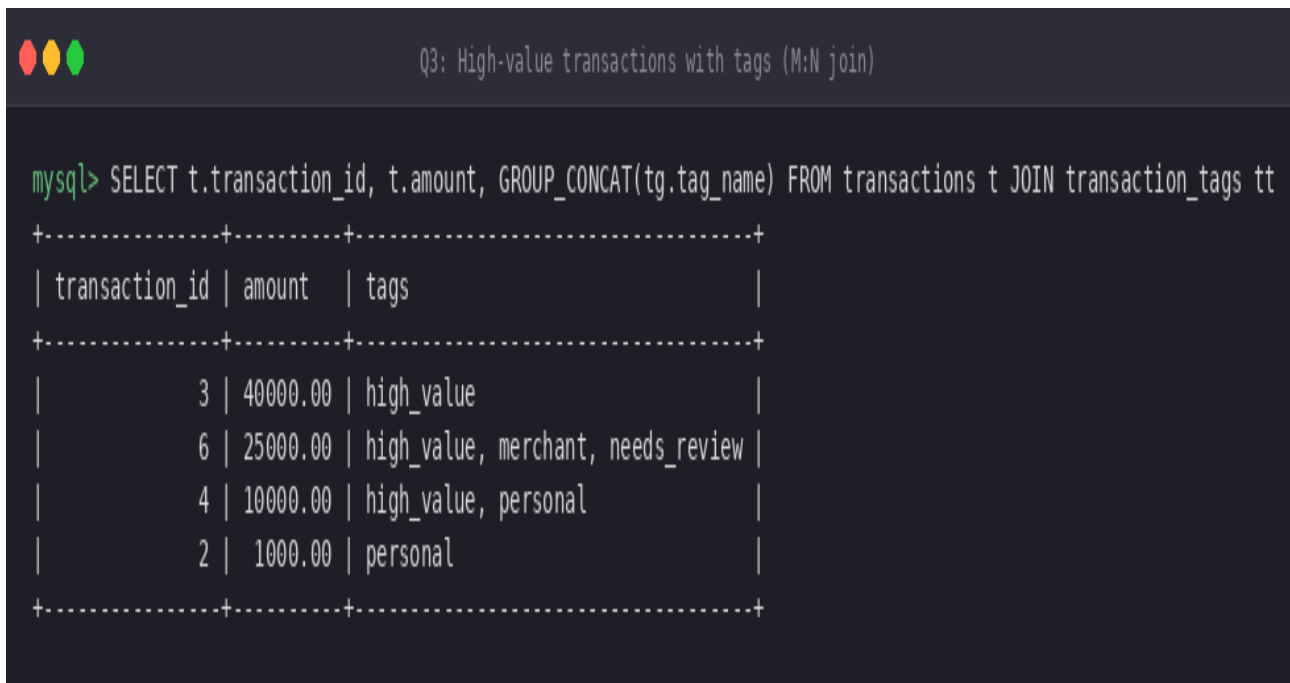


Figure 5 — GROUP_CONCAT across the junction table shows every tag on each high-value transaction.

5.4 ETL warnings and errors (log query)

Q4: Warnings and errors from system_logs

```
mysql> SELECT * FROM system_logs WHERE log_level IN ('WARNING','ERROR');
```

log_id	log_level	source	message
6	ERROR	parser	Malformed date field in raw record; entry sent to dead-letter queue
5	WARNING	cleaner	Merchant name normalized from "DIRECT PAYMENT LTD " (trailing whitespace
3	WARNING	parser	Skipped 40 non-transaction messages (OTP/system notifications)

Figure 6 — Filtered log query used by the ops dashboard.

6. Security & Accuracy Rules — With Proof

The database rejects invalid data at the storage layer rather than trusting the application to sanitise it. This is a defense-in-depth measure: even if a future ETL bug tries to write bad data, the database refuses it. The following rules are enforced.

6.1 Rules enforced

Rule	Mechanism	Location	Purpose
amount > 0	CHECK	transactions	Reject negative or zero amounts
fee ≥ 0	CHECK	transactions	Reject negative fees
new_balance ≥ 0 or NULL	CHECK	transactions	Balance cannot go negative
sender ≠ receiver	TRIGGER	transactions	A user cannot pay themselves
Phone format	CHECK regex	users	Digits/+/* only
Hex color format	CHECK regex	tags	Reject malformed color codes
Unique tx_id	UNIQUE	transactions	Prevent duplicate ingestion of the same SMS
Unique tag name	UNIQUE	tags	Prevent tag confusion
Unique phone	UNIQUE	users	Prevent duplicate users
FK referential integrity	FOREIGN KEY	all	No orphan transactions or logs

6.2 Proof — invalid inserts are rejected

The screenshot below shows the database rejecting five different kinds of invalid data with the correct error codes and messages.



Security & accuracy constraints (rejected invalid data)

```
mysql> INSERT INTO transactions (category_id, amount, tx_datetime) VALUES (1, -500.00, NOW());
```

```
INSERT INTO transactions (category_id, amount, tx_datetime) VALUES (1, -500.00, NOW())
```

```
ERROR 4025 (23000) at line 1: CONSTRAINT `chk_tx_amount_positive` failed for `momo_analytics`.`transactions`
```

```
mysql> INSERT INTO transactions (category_id, sender_id, receiver_id, amount, tx_datetime) VALUES (3, 2, 2, 1000, NOW());
```

```
INSERT INTO transactions (category_id, sender_id, receiver_id, amount, tx_datetime) VALUES (3, 2, 2, 1000, NOW())
```

```
ERROR 1644 (45000) at line 1: Sender and receiver must be different users
```

```
mysql> INSERT INTO users (full_name, phone_number) VALUES ('Test', 'not-a-phone');
```

```
INSERT INTO users (full_name, phone_number) VALUES ('Test', 'not-a-phone')
```

```
ERROR 4025 (23000) at line 1: CONSTRAINT `chk_users_phone_format` failed for `momo_analytics`.`users`
```

```
mysql> INSERT INTO tags (tag_name) VALUES ('high_value');
```

```
INSERT INTO tags (tag_name) VALUES ('high_value')
```

```
ERROR 1062 (23000) at line 1: Duplicate entry 'high_value' for key 'uq_tag_name'
```

Figure 7 — Every constraint fires when violated. Error codes 4025 (CHECK), 1644 (trigger), 1062 (unique), 1452 (FK) all confirm the rules are active.

7. Conclusion

The database delivered here is normalised, constraint-protected, indexed for the queries the dashboard will run, and shipped with a runnable SQL script plus proof of correctness. It supports the assignment 3 REST API directly: the JSON schemas in `examples/json_schemas.json` show how each entity will be serialised, and the `complete_transaction` object mirrors the response shape the `GET /transactions/{id}` endpoint will produce.

8. AI Usage Statement

AI (Claude by Anthropic) was used as a productivity aid: drafting SQL boilerplate, clarifying MySQL/MariaDB syntax differences, generating the ERD image from my chosen schema, and tightening documentation wording. The schema design — the six entities, the M:N choice, which columns to constrain, which indexes to add — was driven by my own analysis of the provided XML dataset. The full log is at `AI_USAGE_LOG.md` in the repository.